Code Implementation of Forward Kinematics and Other Methods

Summary:

In this lesson, students turn the DH steps from Lesson 4 into Python code, one small function at a time. On Day 1, they write arrow, scale, add_vectors, turn_90, turn and flat_tip for the flat arm, and check each function against the answers from Lesson 4. They also hunt for three common bugs. On Day 2, they write arrow_3d and turned_3d for the base turn, reuse the same steps in tip_3d to find the 3D position, check it in the Desmos graph from Lesson 4, and compare it with Lesson 3's method written as code. Then they use loops to try many poses and see where the tip can reach. At the end, every student has a short Python program that finds the end effector position from the joint angles. Students need intermediate Python: variables, functions, lists, for loops and import.

Objective:

By the end of this lesson, students will be able to:

- Use math.radians with math.sin and math.cos to work with angles in degrees
- Write a function that returns a link's arrow (cos θ, sin θ) from its angle
- Write functions that multiply a vector by a number and add two vectors with a for loop
- Turn an arrow 90° and by any angle in code, with the rules from Lesson 4
- Build flat_tip and tip_3d from these small functions, and check them against the answers from Lesson 4
- Find a bug by checking code against an answer they already know
- Compare two methods in code (Lesson 3's trigonometry and Lesson 4's DH steps) and change Lesson 3 angles into DH angles
- Use loops to try many poses and describe the region the tip can reach

Standards:

NGSS

- Engineering Design: HS-ETS1-2 (break a complex problem into smaller problems), HS-ETS1-4 (use a computer model to test a solution)
- Science & Engineering Practices: Developing and Using Models; Using Mathematics and Computational Thinking; Analyzing and Interpreting Data
- Crosscutting Concepts: Systems and System Models; Patterns

Vocabulary:

Function: A named block of code. Numbers go in and one answer comes back out. In this lesson, arrow, turn and flat_tip are functions.

Return: The word that sends the answer out of a function. The value after return is the output.

List: An ordered group of numbers in square brackets, like [176.78, 176.78]. We use a list for a vector.

Index: The position of a number in a list. Python starts counting at 0, so v[0] is the first number and v[1] is the second.

Loop: Code that repeats. A for loop repeats once for each item, such as each position in a list.

Module: A set of ready-made tools. import math gives us math.sin, math.cos, math.sqrt and math.radians.

Radians: A unit for angles where 180° = π (about 3.14159). Python's sin and cos use radians, so math.radians changes degrees into radians first.

Arrow (unit vector): A vector with length 1. arrow(theta) returns [cos θ, sin θ]. Multiply it by a link's length to get the link vector.

Unpacking: Putting the numbers of a list into separate names. x, z = [250.0, 0.0] makes x = 250.0 and z = 0.0.

Bug: A mistake in code that makes the answer wrong.

Reachable region: All the points the tip of the arm can reach.

Motor tick: One count of a servo's position. The servos used here count 4096 ticks for one full turn.

Resources & Materials:

- Lesson 5 slideshow, Days 1 and 2 (36 slides; the slide numbers in the Procedures are from this slideshow)
- Computers with Python 3, one per pair. IDLE, Thonny or any other editor works. No extra libraries are needed (matplotlib is only for an optional plot)
- Student files: arm_starter.py (saved as arm.py), reach_sweep.py, and the optional reach_plot.py and ticks_demo.py
- Teacher solution: arm.py (the same code is on the slides, so hold the file back until students have tried it themselves)
- The Desmos graph from Lesson 4, for checking answers on Day 2
- Students' link tables from Lesson 4, for comparing answers
- The assembled 4-DOF arm from Lesson 2, powered and connected to a computer, to read motor ticks for the optional last part of Day 2 (optional)
- All code uses link lengths L1 = L2 = 250 mm, as in Lessons 3 and 4, and the DH angles from Lesson 4: θS from the floor, θE from the line of the upper arm, and up is +. If the built arm measures differently, change the two constants at the top of arm.py. The answers on the slides will change too.

Procedures:

Day 1: The Flat Arm in Python

- **Opening (4 min)** — Show the title slide and "Where Lesson 4 Left Off" (slides 1–2). In Lesson 4, a link's arrow was (cos θ, sin θ), turning an arrow 90° was (x, y) → (−y, x), the forearm's arrow was f = cos θE · e + sin θE · n, and the tip was L1 e + L2 f. For θS = 45° and θE = −45° the tip was (426.78, 176.78), and with the base at θB = 45° it was (301.78, 301.78, 176.78). Today each of these steps becomes a small Python function, and we test each one against the answers from Lesson 4.
- **Setting up (5 min)** — Show "Setting Up" (slide 3). Students open arm_starter.py and save a copy as arm.py. The file has `import math`, the link lengths `L1` and `L2`, and a helper `show` that prints a vector rounded to 2 decimals. Students run it. If nothing prints and there is no error, it is ready. They will add to this file all lesson, one function at a time.
- **Radians (6 min)** — Show "Python's sin and cos Use Radians" (slide 4). `math.sin` and `math.cos` want radians, not degrees, and `math.radians` changes degrees into radians. `round(x, 2)` keeps 2 decimals, because Python shows tiny errors like 0.49999999999999994. Then Try It Yourself #1 (slide 5): students predict, then run `round(math.sin(math.radians(30)), 2)`, `round(250 * math.cos(math.radians(45)), 2)` and `math.sqrt(3**2 + 4**2)`. The answers are 0.5, 176.78 and 5.0 (slide 6). 176.78 is the upper arm vector's number from Lesson 4, and 5.0 is the length of the vector (3, 4).
- **Step 1: arrow (6 min)** — Show "Step 1: A Function for One Arrow" and "Step 1: arrow" (slides 7–8). A function is a recipe with a name: numbers go in and one answer comes out. `arrow` takes an angle theta, measured from the floor, and returns the link's arrow [cos θ, sin θ]. Students type it in and check `show(arrow(45))`, which prints [0.71, 0.71], and `show(arrow(90))`, which prints [0.0, 1.0]. arrow(45) is e, the upper arm's arrow from Lesson 4 (show rounds 0.7071 to 0.71).
- **Step 2: scale and add_vectors (8 min)** — Show "Step 2: Multiplying and Adding Vectors" and "Step 2: scale and add_vectors" (slides 9–10). To multiply a vector by a number, multiply each number. To add two vectors, add the numbers in the same position. Both functions use a loop with a position number i: i = 0 is the first number and i = 1 is the second. Students type both functions and check `show(scale(250, arrow(45)))`, which gives [176.78, 176.78], the upper arm vector from Lesson 4, and `print(add_vectors([3, 4], [2, -1]))`, which gives [5, 3]. Point out that the loops will work with three numbers tomorrow.
- **Try It Yourself #2 (4 min)** — Students use scale and arrow to print the link vector for L = 250 mm at θ = 30° and at θ = 90° (slide 11). The answers are [216.51, 125.0] and [0.0, 250.0] (slide 12). Python drops trailing zeros, so 125.00 prints as 125.0. These are the answers to Try It Yourself #1 in Lesson 4.
- **Step 3: turning arrows (7 min)** — Show "Step 3: Turning Arrows" (slide 13). `turn_90` is the Lesson 4 rule (x, y) → (−y, x). `turn` is the forearm rule from Lesson 4: it turns the arrow e by an angle toward n, cos θ · e + sin θ · n, using scale and add_vectors. Students check `show(turn_90([3, 4]))`, which gives [-4.0, 3.0], and `show(turn([1, 0], [0, 1], 30))`, which gives [0.87, 0.5]. With e = [1, 0] and n = [0, 1], turn gives the unit circle: [cos 30°, sin 30°].
- **Step 4: flat_tip (7 min)** — Show "Step 4: The Flat Arm" and "Run the Worked Example" (slides 14–15). Each line of flat_tip is one row of the Lesson 4 link table: e = arrow(θS), n = turn_90(e), f

= turn(e, n, θE), and the tip is L1 e + L2 f. Students check `show(flat_tip(45, -45))`. It should print [426.78, 176.78], the same answer as Lessons 3 and 4. Remind students that the elbow bends down, so θE is −45, not 45. Walk through the values inside: e = [0.71, 0.71], n = [-0.71, 0.71] and f = [1.0, 0.0].

- **Try It Yourself #3 (5 min)** — Students run flat_tip for θS = 60°, θE = −30° and for θS = 30°, θE = 0° (slide 16), and compare with their link tables from Lesson 4 (Try It Yourself #2 and #3). The answers are [341.51, 341.51] and [433.01, 250.0] (slide 17).
- **Bug hunt (6 min)** — Show "Bug Hunt" (slide 18). Three broken versions give a wrong answer for θS = 45°, θE = −45° (the right answer is [426.78, 176.78]). In A, math.radians is missing in arrow, so the answer is [374.62, 270.28]. In B, turn_90 turns the arrow clockwise, so the answer is [176.78, 426.78]. In C, the return line in add_vectors is inside the loop, so the answer is [426.78] with only one number. Pairs find the mistake in each, then show "Bug Hunt: Answers" (slide 19). Teaching point: check new code against an answer you already know. Note that the flat pose, flat_tip(0, 0), does not catch bugs A and B, because both still give [500.0, 0.0].
- **Recap (2 min)** — Show "Day 1 Recap" (slide 20). arrow gives a link's arrow, scale and add_vectors multiply and add vectors, turn_90 and turn turn an arrow, and flat_tip puts them together. Tell students that tomorrow the base turns, so every vector gets a third number.

Day 2: The Base Turn, Two Methods and the Arm's Reach

- **Recap (3 min)** — Show "Day 2: Where Day 1 Left Off" (slide 21). Ask: What does arrow(90) return? ([0.0, 1.0].) What does turn_90([0.6, 0.8]) return? ([-0.8, 0.6].) What does flat_tip(45, -45) return? ([426.78, 176.78].)
- **Step 5: the arrows in 3D (8 min)** — Show "Step 5: The Arrows in 3D" and "Test the 3D Arrows" (slides 22–23). `arrow_3d` is e from Lesson 4: (cos θS cos θB, cos θS sin θB, sin θS). Both angles are changed to radians. `turned_3d` is n. Lesson 4 checked that n is e with θS + 90° in place of θS, so turned_3d just calls arrow_3d with theta_s + 90. Students check `show(arrow_3d(45, 45))`, which gives [0.5, 0.5, 0.71], `show(turned_3d(45, 45))`, which gives [-0.5, -0.5, 0.71], and `show(scale(250, arrow_3d(45, 45)))`, which gives [125.0, 125.0, 176.78]. These are e, n and the upper arm vector in the worked example in Lesson 4.
- **Step 6: tip_3d (5 min)** — Show "Step 6: The Whole Arm in 3D" (slide 24). Compare tip_3d with flat_tip: only the lines for e and n changed. turn, scale and add_vectors need no changes, because their loops work for any number of entries. This is the DH idea from Lesson 4: the forearm only needs the upper arm's arrows, e and n.
- **Run the worked example and check in Desmos (4 min)** — Show "Run the Worked Example" (slide 25). `show(tip_3d(45, 45, -45))` prints [301.78, 301.78, 176.78], the same answer as Lesson 4. In the Desmos graph from Lesson 4, set u = 45, v = 45 and w = −45. Then d reads (3.0178, 3.0178, 1.7678), which is (301.78, 301.78, 176.78) mm after multiplying by 100. Here u = θB, v = θS and w = θE.
- **Try It Yourself #4 (6 min)** — Students run tip_3d for θB = 30°, θS = 30°, θE = 0° and for θB = 30°, θS = 60°, θE = −90° (slide 26). These are Try It Yourself #4 and #5 from Lesson 4. The answers are [375.0, 216.51, 250.0] and [295.75, 170.75, 91.51] (slide 27). Students compare with their link tables and check in Desmos (d = (3.7500, 2.1651, 2.5000) and (2.9575, 1.7075, 0.9151)).
- **Another method: Lesson 3 in code (6 min)** — Show "Another Method: Lesson 3 in Code" (slide 28). The function `trig_tip` uses Lesson 3's angles and triangles: θ1 is measured from straight

up, θ2 is the elbow bend and θ3 is the base turn. The reach is r = L1 sin θ1 + L2 sin(θ1 + θ2), the height is z = L1 cos θ1 + L2 cos(θ1 + θ2), and the base turn gives x = r cos θ3 and y = r sin θ3. Lesson 3 called the height Y; here it is z, as in Lesson 4.

- **Do the two methods agree? (5 min)** — Show "Do the Two Methods Agree?" (slide 29). The poses are in Lesson 3's angles. tip_3d needs DH angles, so the loop uses the rule from Lesson 4: θB = θ3, θS = 90° − θ1 and θE = −θ2. For each pose it prints the answer from trig_tip and from tip_3d, and the two lines match each time. The three poses are the worked example and Try It Yourself #4 and #5. Ask: Why is it useful to have two methods? (If they ever differ, one of them has a bug.)
- **Where can the tip go? (4 min)** — Show "Where Can the Tip Go?" (slide 30). A function can be used many times, and a loop can try every pose. θS goes from 0° to 90° and θE from −90° to 0° (the elbow bending down), in steps of 10°. `range(-90, 1, 10)` gives −90, −80 … 0 (it stops before 1). We keep θB = 0°, so we see the arm from the side.
- **Sweeping the angles (7 min)** — Show "Sweeping the Angles" (slide 31) and walk through reach_sweep.py. One loop sits inside another, so every pair of θS and θE is tried. `x, z = flat_tip(...)` takes the two numbers out of the list, and `math.sqrt(x ** 2 + z ** 2)` is the distance from the base. The program prints 100 poses, farthest 500.0 mm and nearest 353.55 mm. Then show "The Reachable Region" (slide 32): each dot is one pose. Turning the base spins this picture around the z axis. These angle ranges are for practice and are not the real arm's limits. If matplotlib is installed, students can run reach_plot.py to draw the same picture.
- **Try It Yourself #5 (5 min)** — Students change the elbow loop to `range(-150, 1, 10)` and predict whether the nearest distance gets bigger or smaller (slide 33). Then they run it. The answers are 160 poses, farthest 500.0 mm and nearest 129.41 mm (slide 34). 129.41 is 2 × 250 × cos 75°: with θE = −150°, the bent elbow brings the tip much closer to the base.
- **Optional: from motor ticks to angles (3 min)** — If time allows, show "Optional: From Motor Ticks to Angles" (slide 35). The servos count 4096 ticks for one full turn, so 1 degree is 4096 ÷ 360 ticks, and the base motor is geared 3 to 1. Counting from ZERO_TICKS, 512 ticks is 45° on the shoulder or the elbow, and 1536 ticks is 45° on the base. ZERO_TICKS is the reading when the joint is at 0° and must be measured on the built arm (2048 in ticks_demo.py is a placeholder). If a motor counts the other way, change the sign so that up is +, the DH way.
- **Wrap-up (4 min)** — Show "What We Learned" (slide 36). Each DH step from Lesson 4 became a small function, and tip_3d reused turn, scale and add_vectors without changes. Checking against an answer we know finds bugs. Two methods gave the same answer, and loops can map the arm's reach. Preview Lesson 6: inverse kinematics goes the other way, from a target position back to the joint angles.

Assessment:

Assess with working code. On Day 1, each student pair shows flat_tip printing [426.78, 176.78] for 45° and −45°, and their answers to Try It Yourself #2 and #3. On Day 2, pairs show tip_3d matching the Lesson 4 answers (#4) and the output of reach_sweep.py. Students check each answer against the solution on the slide, and can check #2 to #5 by running their own code.

Use these questions as an exit ticket or short quiz. The answers are in red.

- Why must the code change degrees into radians before using math.sin and math.cos?

Answer: Python's sin and cos use radians. If they are given degrees, Python reads 45 as 45 radians and the answer is wrong ([374.62, 270.28] for flat_tip(45, -45)).

- What does arrow(30) return, and what does the input mean? How do you get the link vector for a 250 mm link?
  Answer: [0.87, 0.5]. theta = 30 is the link's angle from the floor in degrees, and the arrow has length 1. scale(250, arrow(30)) gives the link vector [216.51, 125.0].

- In add_vectors, what does range(len(u)) do?
  Answer: It counts 0, 1, 2 … once for each number in u. The number i is the position, so u[i] + v[i] adds the numbers in the same position. It works for 2 or 3 numbers.

- A student writes turn_90 as return [v[1], -v[0]]. What goes wrong?
  Answer: It turns the arrow 90° clockwise instead of counterclockwise, so n points the wrong way. flat_tip(45, -45) gives [176.78, 426.78] instead of [426.78, 176.78]. The rule is (x, y) → (−y, x).

- A student's add_vectors gives [426.78] for flat_tip(45, -45), with only one number. What is wrong?
  Answer: The return line is inside the loop, so the function stops after the first number. Move return back so it lines up with for.

- Why did tip_3d not need a new turn function?
  Answer: turn only uses scale and add_vectors, and their loops work for any number of entries. In 3D only e and n change (arrow_3d and turned_3d). The forearm still only needs the upper arm's arrows: the DH idea.

- Lesson 3's angles θ1 = 60°, θ2 = 0°, θ3 = 30° describe a pose. Which call to tip_3d gives the same tip?
  Answer: tip_3d(30, 30, 0), because θB = θ3 = 30°, θS = 90° − θ1 = 30° and θE = −θ2 = 0°. Both give [375.0, 216.51, 250.0].

- Name three ways to check that tip_3d(45, 45, -45) = [301.78, 301.78, 176.78].
  Answer: The link table by hand (Lesson 4), the Desmos graph with u = 45, v = 45, w = −45 (d = (3.0178, 3.0178, 1.7678), times 100), and trig_tip(45, 45, 45) (Lesson 3's method in code).

- The sweep tries θS from 0° to 90° and θE from −90° to 0° in steps of 10°. How many poses is that, and why is the farthest distance exactly 500.0 mm?
  Answer: 10 × 10 = 100 poses (each range gives 10 values). The farthest tip is at θE = 0°, when the forearm continues straight on from the upper arm: 250 + 250 = 500 mm.